

Automation Studio

Prototype · version 0.1.0

System architecture, operation principles, codebase reference and operating procedures.

Document	Automation Studio: Technical Description and Operation
System version	0.1.0 (prototype). The code is frozen at this version.
Date	25 September 2026
Repository	https://github.com/Ajinkya2101/Automation-Studio
Writing standard	ASD-STE100 Simplified Technical English rules
Code size	Approximately 3,200 lines: Python, JavaScript, HTML/CSS, shell and batch

NOTE: This document describes the prototype. The prototype does not connect to Oracle Fusion. It uses the Acme Mail demo application and Outlook on the web as target applications.

Contents

1. Introduction	4
1.1 Purpose of the system	4
1.2 Scope of this document	4
1.3 Writing rules	4
1.4 Terms	4
2. System architecture	6
2.1 Overview	6
2.2 Components	7
2.3 Deployment	7
3. Operation principles	8
3.1 Test scripts in Excel	8
3.2 Sign-in session	8
3.3 Recording	8
3.4 Parameters and tokens	9
3.5 Review of a recording	9
3.6 Automated run	10
3.7 Locator resolution	11
3.8 Live events and live view	11
3.9 Results	12
4. Data and storage	13
4.1 Recorded action	13
4.2 Run record	13
5. Interfaces	13
5.1 REST API	13
5.2 Live event types	14
5.3 Settings	14
6. Codebase reference	15
7. Operating procedures	16
7.1 Start the system	16
7.2 Sign in to Outlook on the web	16
7.3 Record a test	16
7.4 Run a test	16
7.5 Stop, rebuild or remove the container	17
7.6 Run the tests	17

7.7 Start without Docker	17
8. Safety and security precautions	17
9. Limits and known problems	18
10. Tests	18

1. Introduction

1.1 Purpose of the system

Automation Studio records a test that a person does in a web application. Then it does the same test again automatically, as many times as necessary. It records each action against a step of an Excel test script. During an automated run, it writes a log line for each action and a screenshot for each step. At the end of the run, it writes the results into a copy of the Excel workbook.

The prototype shows the method before the team uses it on Oracle Fusion. The method is: write the test in Excel, record it one time, then run it automatically after each change.

1.2 Scope of this document

This document gives:

- The system architecture (Section 2).
- The operation principles: how the system reads, records and runs a test (Section 3).
- The data and storage layout (Section 4).
- The interfaces: REST API, live events and settings (Section 5).
- A reference for each file in the codebase (Section 6).
- The operating procedures (Section 7), the safety and security precautions (Section 8), the limits (Section 9) and the tests (Section 10).

1.3 Writing rules

This document uses the ASD-STE100 Simplified Technical English rules. Procedures use short sentences in the imperative. Each sentence gives one instruction. Descriptions use short sentences in the active voice. Names of files, functions, settings and screen controls are technical names. They are shown as they occur in the code or on the screen.

1.4 Terms

Term	Meaning in this document
Target application	The web application that the test uses, for example Outlook on the web or Acme Mail.
Test case	One sheet of the Excel workbook, for example TS_OUT001.
Step	One row of a test case, for example TS_OUT001.3. A step has a description, test data and an expected result.
Test data	The Key: value lines in the Test Data cell of a step, for example Subject: Automation Studio test {now}.
Token	A placeholder in test data that the system replaces at the start of each session: {now}, {date}, {time}, {stamp}.
Action	One recorded operation: open, click, type, press a key, select, or check.
Check	An action that makes sure that a text shows on the screen. The operator adds it with + Add check.
Parameter	A reference such as \${Subject} in a recorded action. The system replaces it with test data during a run.
Locator candidate	One way to find an element on the page, for example by role and name, by label, or by CSS selector.

Term	Meaning in this document
Recording	The saved list of actions for each step of a test case.
Run	One automated execution of a recording, with its log, screenshots and results workbook.
Session	One sign-in, recording or run. Only one session can operate at a time.
Browser profile	A folder that keeps cookies and storage for one site. It keeps the sign-in between sessions.
Live view	The dashboard panel that shows the browser inside the container, through noVNC.

2. System architecture

2.1 Overview

The system operates in one Docker container on the Windows computer of the operator. The operator uses a web browser to control the system. The browser shows two pages from the container: the dashboard on port 8010 and the live view on port 6080.

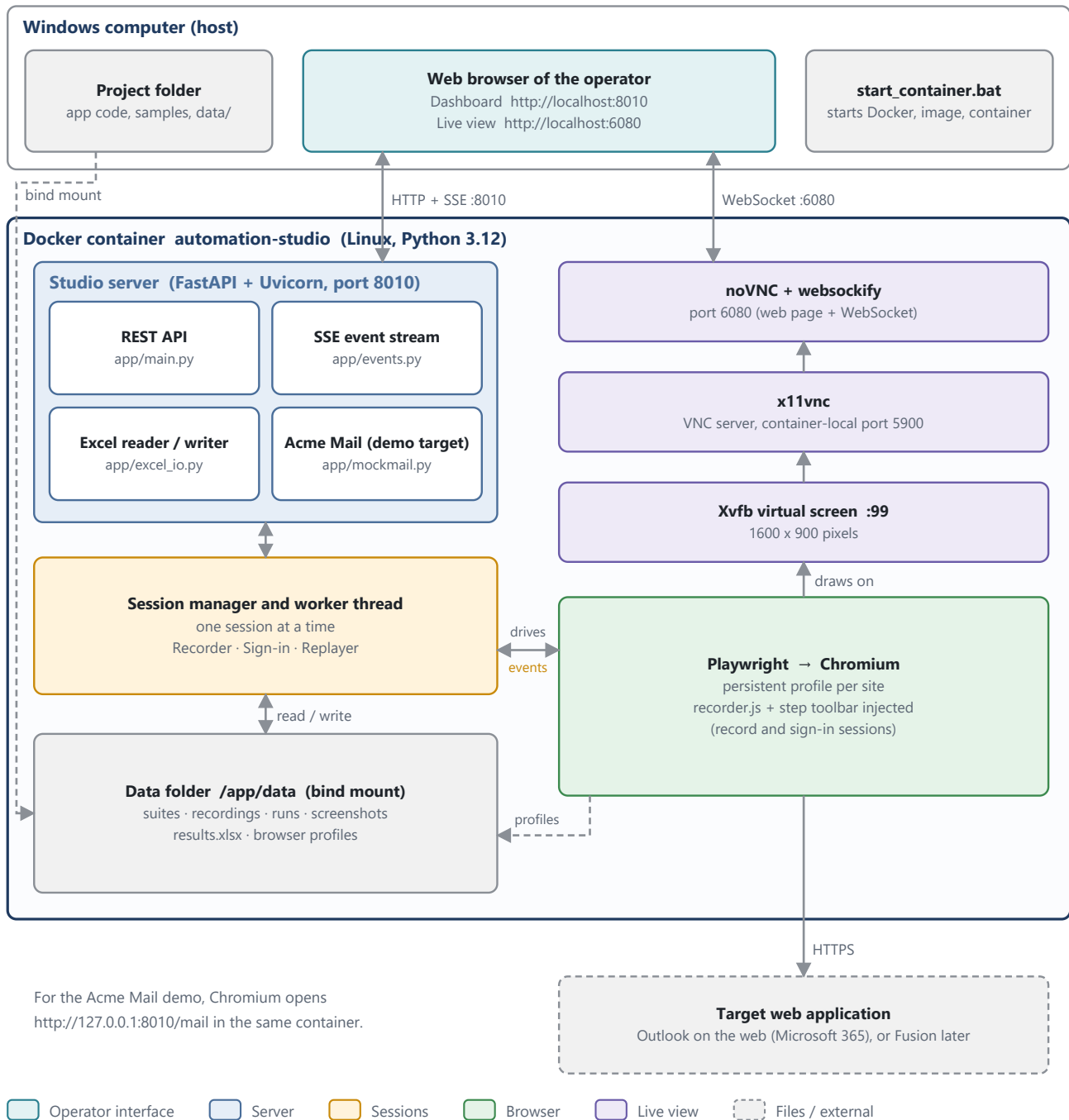


Figure 1: System architecture. All components in the large frame operate in the container.

2.2 Components

Component	Location	Function
Dashboard	app/static/	Shows test cases, steps, recordings and runs. Starts sessions. Shows live events and the live view.
Studio server	app/main.py	Gives the REST API and the event stream. Serves the dashboard and the Acme Mail demo.
Excel reader and writer	app/excel_io.py	Reads test cases from the workbook. Writes results into a copy of the workbook.
Session manager	app/events.py	Starts one session at a time on a worker thread. Keeps the event log of each session.
Recorder	app/recorder.py, app/recorder.js	Opens Chromium, injects the step toolbar, receives the actions of the operator and saves the recording. Also operates the sign-in session.
Replayer	app/replayer.py	Does the recorded actions again with the current test data. Makes the log, screenshots and results.
Browser launcher	app/browser.py	Starts Chromium with a persistent profile for each site.
Action helpers	app/actions.py	Tokens, parameters, start URL and the text of each action in the log.
Storage	app/storage.py	Reads and writes the JSON files in the data folder.
Acme Mail	app/mockmail.py	A small webmail application. It is a safe target for demonstrations and tests.
Live view chain	docker/entrypoint.sh	Xvfb gives a virtual screen. x11vnc shares it. websockify and noVNC show it in a web page.
Launcher	start_container.bat	Starts Docker Desktop, builds the image, starts the container and opens the dashboard.

2.3 Deployment

The launcher creates the container automation-studio from the image automation-studio:latest. The image contains Python 3.12, Playwright, Chromium, Xvfb, x11vnc, websockify and noVNC. The launcher mounts the project folder at /app. Thus the data folder stays on the host when you rebuild the container.

Port	Bound to	Use
8010	127.0.0.1 on the host	Dashboard, REST API and event stream. Acme Mail at /mail.
6080	127.0.0.1 on the host	Live view (noVNC web page and WebSocket).
5900	Container only	VNC server (x11vnc). It is not published.

The launcher also gives these settings to the container: the live view URL, and the time zone of the host as a POSIX offset (for example LOCAL-05:30). Thus the times in the dashboard and in {now} tokens agree with the host clock. The container has a 1 GB shared memory area for Chromium, and the restart policy unless-stopped.

3. Operation principles

3.1 Test scripts in Excel

The reader uses the template of the implementation team. Each sheet is one test case. The reader finds the header row that starts with Test Step ID. It finds the columns by their header text. A row with an ID without a full stop is the test-case row. A row with an ID such as TS_OUT001.3 is a step.

- The Test Data cell gives Key: value lines. A line without a key continues the value on the line before it.
- If the Test Case ID cell has an error value such as `=#REF!`, the reader uses the ID from the step IDs. It gives a warning.
- If a test data key has no value, the reader gives a warning.
- A URL key in the test data sets the target application. If there is no URL, the target is Acme Mail.

3.2 Sign-in session

Some target applications need a sign-in, for example Outlook on the web with multi-factor authentication. The sign-in session opens the target application in the browser profile of that site. The operator signs in in the live view and clicks Save sign-in. The system writes a marker file in the profile. It does not record or keep the password. Recording and run sessions use the same profile, so they start with the sign-in already done.

3.3 Recording

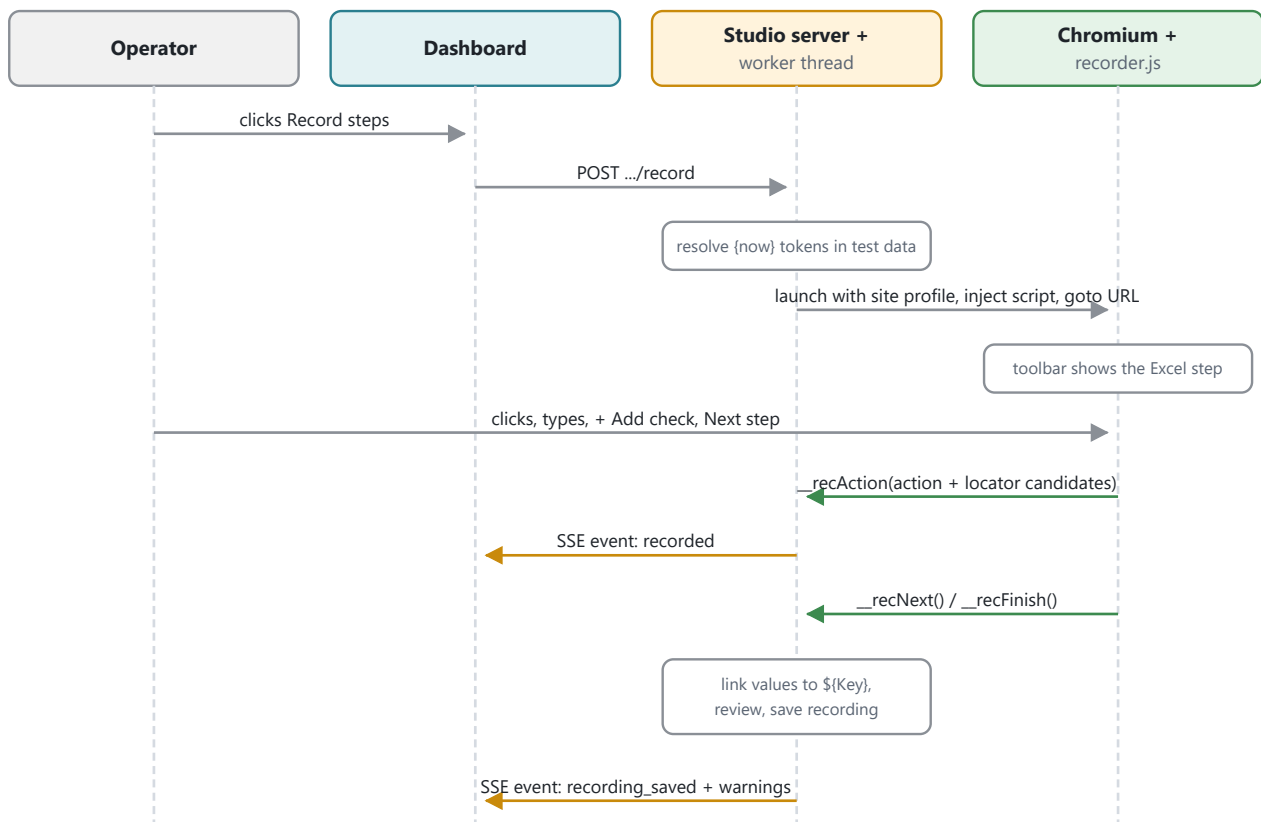


Figure 2: Sequence of a recording session.

The recorder starts Chromium and adds recorder.js to each page before the page scripts start. The script shows a toolbar with the current Excel step, its test data and its expected result. It listens to the events of the page in the capture phase. It sends each action to Python through exposed functions.

The operator does this	The recorder saves this
Clicks a button, link, tab, menu item, option or tree item	click action with locator candidates. It also records if the click loads a new page.
Types in a field or in a rich-text editor (contenteditable)	fill action with the final text of the field.
Pushes Enter in a single-line field	fill action, then press action for Enter.
Selects an item in a drop-down list	select action.
Clicks + Add check, then clicks a text on the page	expect_text action with that text.
Clicks Next step or Finish recording	The next actions go to the next step, or the recording stops.

Pending typing

A field normally sends its text when the focus goes to a different element. Some applications prevent this. For example, the Send button of Outlook keeps the focus in the message body, and its editor does not send input events. Thus the recorder also sees typing from key, paste and cut events. Before it records a click, a key, a check or a toolbar action, it saves the pending text of the active field.

Locator candidates

For each element, recorder.js makes a list of candidates, from the most stable to the least stable: test ID, label or placeholder (fields), role and accessible name (controls), visible text, [aria-label] selector and CSS path. It removes icon-font characters from names and text. It does not use element IDs that contain digits, because applications often make new IDs at each page load.

3.4 Parameters and tokens

When the recording stops, the system compares the recorded values with the test data. If a typed value is the same as a test data value, the system replaces it with a parameter such as \${Subject}. If a check text contains test data values, the system replaces each value in the text. Thus a run types the current test data, not the values of the recording.

Tokens make a value different in each session. {now} gives the date and time, {date} the date, {time} the time and {stamp} a compact form. The system replaces tokens at the start of each recording and each run. The toolbar shows the values after replacement, so the operator types the same values.

3.5 Review of a recording

The system examines each recording when it saves it. It shows warnings in the log and on the Recorded automation tab when:

- All actions are in one step, or some steps have no actions.
- No typed value agrees with the test data.
- Some test data values are not used by an action (for example, the message body was not captured).
- A typed value is only the start of a test data value (for example, a recipient selected from suggestions).
- The recording has no checks.

3.6 Automated run

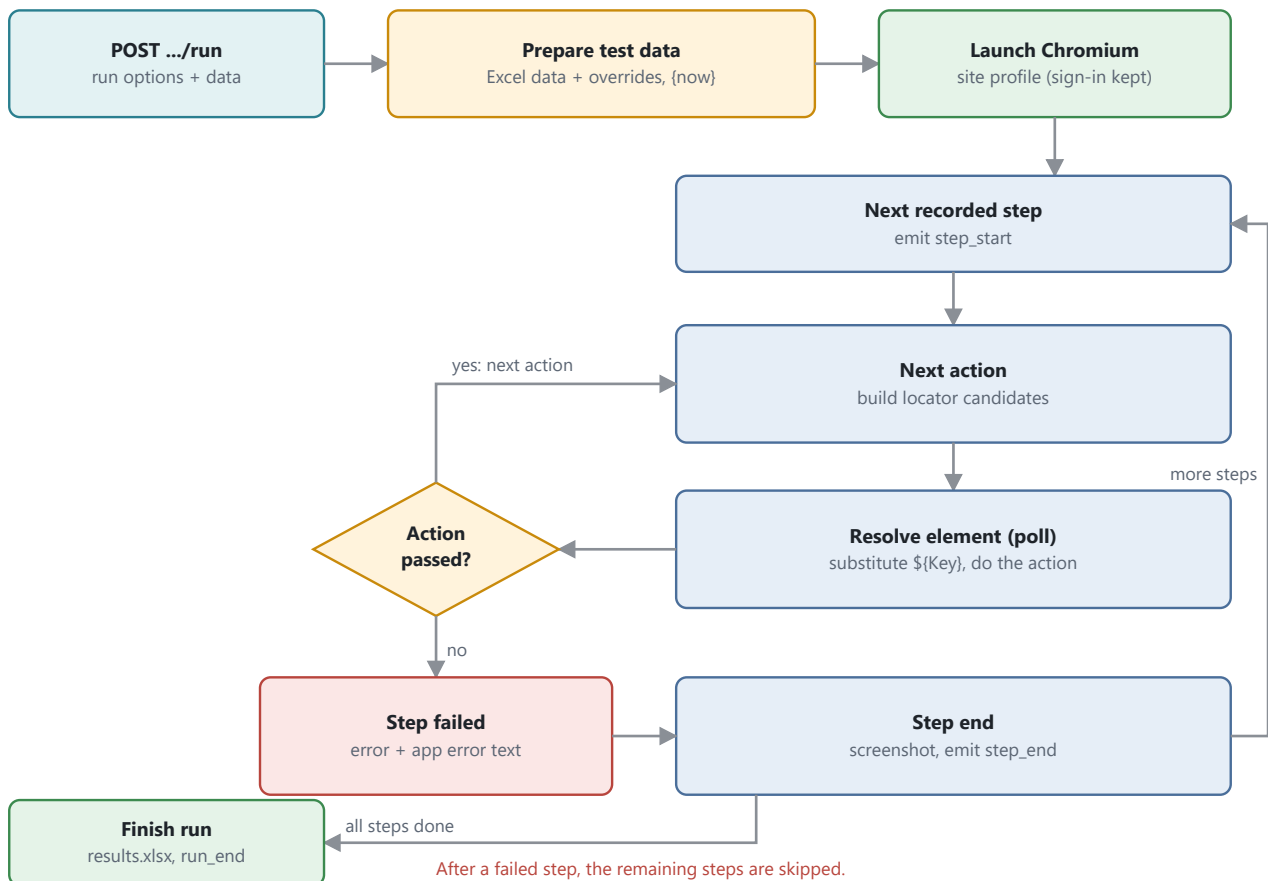


Figure 3: Flow of an automated run.

The replayer prepares the test data: the Excel values, then the values that the operator changed in the run form, then the tokens. It opens Chromium with the profile of the target site. For each action, it finds the element, replaces the parameters and does the action. If a click loaded a new page during the recording, the replayer waits for that page. Before it pushes Enter, it waits 0.6 seconds, so that suggestion lists can show.

If an action fails, the replayer records the error and the error messages that the application shows. It makes a screenshot and marks the remaining steps as skipped. For Acme Mail runs, it can first clear the sent items, so that each run starts from the same data.

3.7 Locator resolution

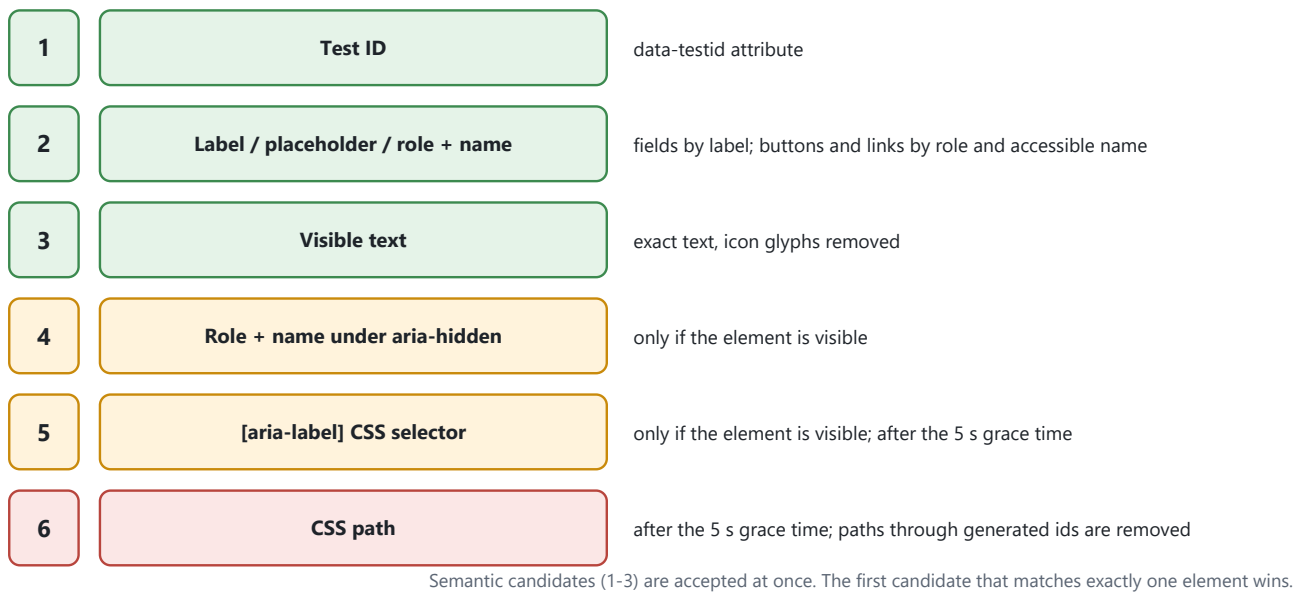


Figure 4: Order of locator candidates during a run.

The replayer examines the candidates again and again until one matches, for a maximum of 15 seconds. It accepts semantic candidates at once. It accepts CSS candidates only after 5 seconds. Thus a CSS path cannot select a wrong element while the page loads.

Some applications set aria-hidden on the full page when a popup is open. Outlook does this. Role locators ignore elements under aria-hidden. Thus the replayer adds two more candidates for each role locator: the same role locator with hidden elements included, and an [aria-label] selector. It uses them only for an element that is visible on the screen.

3.8 Live events and live view

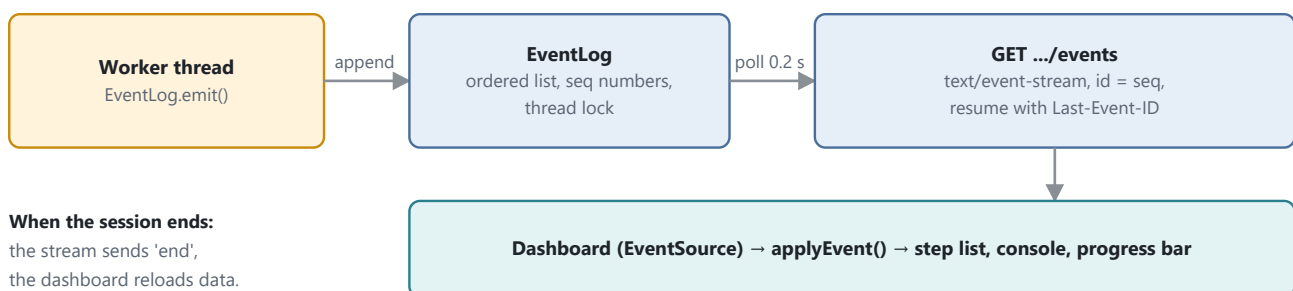


Figure 5: Path of a live event from the worker thread to the dashboard.

The worker thread writes events to the event log of its session. The server sends the log to the dashboard as Server-Sent Events. Each event has a sequence number. If the connection stops, the browser connects again and the server continues after the last number. Thus no event occurs two times.

The live view is independent of the event stream. Chromium shows its window on the Xvfb screen. x11vnc reads the screen, and noVNC shows it in a frame on the dashboard. The operator can use the mouse and keyboard in this frame.

3.9 Results

After a run, the Excel writer copies the uploaded workbook. It writes Observed Results, Status (Pass, Fail or Not Run) and Notes for each step. It writes the tester name and the date. It does not change the original workbook.

4. Data and storage

The system keeps all data as files in the data folder. It writes each JSON file to a temporary file first, then replaces the old file. The data folder is not in the Git repository.

```
data/
|-- suites/<suite-id>/
|   |-- workbook.xlsx           uploaded workbook (not changed)
|   |-- suite.json              test cases, steps, test data, upload warnings
|   |-- recordings/<case-id>.json recorded actions for each step
|-- runs/<run-id>/
|   |-- run.json                status, step results, all events of the run
|   |-- screenshots/NN_<step>.png one screenshot after each step
|   |-- results.xlsx            copy of the workbook with the results
|-- profiles/<site-host>/
|   |-- .studio-signin          Chromium profile for each site (cookies, storage)
|                               time of the last "Save sign-in"
```

4.1 Recorded action

Each action in a recording has this form. The replayer uses the locators in the order that Section 3.7 gives.

```
{
  "type": "fill",                open (goto) | click | fill | press | select | expect_text
  "target": "field \"Add a subject\"", text for the log
  "value": "${Subject}",        parameter or literal value
  "param": "Subject",           test data key(s) linked to this action
  "masked": false,              true for password fields: the log shows *****
  "navigates": false,           true if the click or key loaded a new page
  "locators": [
    {"kind": "label", "value": "Add a subject"},
    {"kind": "css", "value": "input[aria-label=\"Add a subject\"]"}
  ]
}
```

4.2 Run record

run.json keeps the status of the run (running, passed, failed or stopped), the start and end times, the counts of passed, failed and skipped steps, and a result for each step: status, observed text, duration, screenshot and error. It also keeps all events of the run, so the dashboard can show the log of an old run.

5. Interfaces

5.1 REST API

Method	Path	Function
GET	/api/health	Health check. The launcher and the Docker health check use it.
GET	/api/status	The active session, if there is one.
GET	/api/config	Live view URL and container flag for the dashboard.
GET	/api/sample?name=acme outlook	Downloads a sample workbook.
GET	/api/cases	All test cases, with recording status and last run.
POST	/api/suites	Uploads a workbook (.xlsx) and reads its test cases.
GET	/api/suites/{suite}/cases/{case}	Test case, target, sign-in status, recording and runs.

Method	Path	Function
POST	/api/suites/{suite}/cases/{case}/signin	Starts a sign-in session.
POST	/api/suites/{suite}/cases/{case}/record	Starts a recording session.
POST	/api/suites/{suite}/cases/{case}/run	Starts a run. Body: headed, slow_mo, reset_mailbox, data.
POST	/api/sessions/{session}/stop	Stops a session.
GET	/api/sessions/{session}/events	Server-Sent Events of a session.
GET	/api/runs/{run}	Run record with its events.
GET	/api/runs/{run}/results	Downloads the results workbook.
POST	/api/mailbox/reset	Clears the Acme Mail sent items.

A request that starts a session when a different session operates gets the status 409 (conflict).

5.2 Live event types

Event	Sent when
recording_started, recorded, step_change, recording_saved	A recording starts, captures an action, goes to the next step, and is saved.
signin_started, signin_saved	A sign-in session starts, and the operator saves the sign-in.
run_start, step_start, action, step_end, step_skipped, run_end	A run starts, a step starts, an action ends (pass or fail), a step ends, a step is skipped, and the run ends.
info, warning, error	General messages, recording review warnings, and unexpected errors.
session_end	The session ends. Then the stream sends end and closes.

5.3 Settings

Environment variable	Default	Function
STUDIO_PORT	8010	Port of the server.
STUDIO_HOST	127.0.0.1 (0.0.0.0 in the container)	Address that the server listens on.
STUDIO_DATA_DIR	./data	Data folder. The tests use a temporary folder.
STUDIO_LIVE_VIEW_URL	empty (set by the launcher)	noVNC URL for the dashboard frame. If empty, the browser opens as a window.
STUDIO_IN_CONTAINER	empty (1 in the container)	Sets the browser window size for the virtual screen.
STUDIO_SCREEN	1600x900	Size of the virtual screen and browser window.
STUDIO_ACTION_TIMEOUT_MS	15000	Maximum time to find an element or to see a check text.
TZ	host offset (set by the launcher)	Time zone of the container.

6. Codebase reference

File	Contents
run.py	Starts Uvicorn with the FastAPI app. Sets the Proactor event loop on Windows. Opens the dashboard when it is not in a container.
app/config.py	Paths, ports, live view URL, screen size and action timeout. Environment variables change them.
app/main.py	FastAPI app: REST API (Section 5.1), static files, the /artifacts folder and the Acme Mail router.
app/events.py	EventLog (thread-safe list of events, SSE generator with resume) and SessionManager (one session at a time, worker thread, stop flag).
app/storage.py	IDs, time stamps with UTC offset, atomic JSON write, and the paths of suites, recordings and runs.
app/excel_io.py	parse_workbook(): header search, column aliases, test data lines, warnings. write_results(): results into a copy.
app/actions.py	resolve_tokens(), resolved_steps(), start_url(), substitute(), parameterize() and describe().
app/browser.py	profile_dir(), open_context() (persistent context, CSP bypass, window size), sign-in marker, removal of old profile locks.
app/recorder.py	record_session(): bindings, navigation detection, step state, parameters, review warnings, save. signin_session().
app/recorder.js	In-page script: event capture, pending typing, locator candidates, check mode, step toolbar and sign-in toolbar (shadow DOM).
app/replayer.py	run_session(): data, browser, steps, actions, screenshots, results. _candidates() and _resolve() (Section 3.7).
app/mockmail.py	Acme Mail: login, inbox, compose, send (0.8 s delay), sent items, message view, reset.
app/static/index.html, app.js, styles.css	Dashboard: upload, case list, steps, recording view, run form, live panel, console, live view frame, run history.
Dockerfile	Image: python:3.12-slim-bookworm, X11 and VNC packages, Playwright Chromium with system dependencies, health check.
docker/entrypoint.sh	Starts Xvfb, x11vnc and websockify, makes the sample workbooks if necessary, then starts run.py.
start_container.bat	Launcher with the commands build, open, shell, stop, remove, logs and status.
samples/make_sample.py	Makes Automation_Email.xlsx (Acme Mail) and Outlook_Email.xlsx (Outlook) in the team template.
tests/smoke_test.py	End-to-end test on Acme Mail (Section 10).
tests/rich_editor_test.py	End-to-end test on a page that copies the behaviour of the Outlook compose form (Section 10).

7. Operating procedures

7.1 Start the system

NOTE: Docker Desktop must be installed on the computer. The first start builds the image. This takes some minutes.

- (1) Open the project folder.
- (2) Double-click start_container.bat.
- (3) Wait until the launcher shows Automation Studio is up.
- (4) Make sure that the dashboard opens at <http://localhost:8010>.

7.2 Sign in to Outlook on the web

- (1) Upload Outlook_Email.xlsx. Drag the file onto Upload test Excel.
- (2) Click Sign in.
- (3) In the live view, sign in to Microsoft. Approve the multi-factor authentication request.
- (4) If the system asks Stay signed in, click Yes.
- (5) When the mailbox shows, click Save sign-in on the toolbar.
- (6) Make sure that the dashboard shows sign-in saved.

7.3 Record a test

CAUTION: When you click Send during a recording in Outlook, the system sends a real email.

- (1) Select the test case in the list.
- (2) Click Record steps.
- (3) Read the step on the toolbar.
- (4) Do the step in the live view. Type the values that the toolbar shows.
- (5) To add a check, click + Add check. Then click the text that shows that the step is correct.
- (6) Click Next step.
- (7) Do steps 3 to 6 again for each step.
- (8) On the last step, click Finish recording.
- (9) Open the Recorded automation tab. Read the warnings, if there are warnings.
- (10) If a warning shows a problem, click Re-record and record the test again.

7.4 Run a test

CAUTION: Each run of TS_OUT001 sends a real email from the Outlook account.

- (1) Click Run automated.
- (2) Change the test data in the run form, if necessary.
- (3) Select Show the browser to see the run in the live view.
- (4) Click Start run.
- (5) Monitor the step list, the console and the live view.
- (6) When the run ends, click Download results Excel.

7.5 Stop, rebuild or remove the container

To do this	Use this command
Stop the container and keep it	<code>start_container.bat stop</code>
Start it again	<code>start_container.bat</code>
Rebuild the image after a change to requirements.txt or the Dockerfile	<code>start_container.bat build</code>
Apply a change to the Python code	<code>docker restart automation-studio</code>
See the server log	<code>start_container.bat logs</code>
Remove the container (the data folder stays)	<code>start_container.bat remove</code>

7.6 Run the tests

- (1) Start the container.
- (2) Type: `docker exec automation-studio python tests/smoke_test.py`
- (3) Make sure that the last line is All smoke checks passed.
- (4) Type: `docker exec automation-studio python tests/rich_editor_test.py`
- (5) Make sure that the last line is All rich-editor checks passed.

7.7 Start without Docker

- (1) Install Python 3.12.
- (2) Type: `python -m venv .venv`
- (3) Type: `.venv\Scripts\python -m pip install -r requirements.txt`
- (4) Type: `.venv\Scripts\python -m playwright install chromium`
- (5) Type: `.venv\Scripts\python run.py`

NOTE: Without Docker, Chromium opens as a window on the desktop. The dashboard does not show a live view.

8. Safety and security precautions

WARNING: Do not put the data folder in a repository or share it. The profiles folder contains the session cookies of the Outlook account. A person with these files can open the mailbox.

CAUTION: Do not publish ports 8010 and 6080 on a network interface. The live view has no password. A person with access to it can use the signed-in browser.

CAUTION: Do not record a password that is not in the test data. The recorder keeps it as a literal value.

CAUTION: Make sure that a test for a real application uses test accounts and test data. The actions of a run occur in the real application.

9. Limits and known problems

Limit	Effect
One session at a time	A second sign-in, recording or run must wait. The API gives status 409.
One operator	The dashboard has no user accounts and no access control.
Page structure of the target application	If the application changes its page structure, a run can fail. Record the test again.
Elements in frames	The recorder captures actions in frames, but the replayer finds elements only in the main page.
Session expiry	If Microsoft stops the sign-in, step 1 fails. Do the sign-in again.
Recording by a person	The system does not convert Excel steps to actions automatically. This is the next stage of the project.

10. Tests

Test	What the test makes sure of
tests/smoke_test.py	Upload and read of the sample workbook. Recording of five steps by a simulated operator. Links to test data. Detection of clicks that load a page. Run with five passed steps and five screenshots. Results workbook with Pass. Run with a different Subject. Run with a wrong recipient: step 4 fails, step 5 is skipped, and the log shows the application error.
tests/rich_editor_test.py	A page that copies Outlook: a recipient box that makes a chip on Enter, an editor without input events, a Send button that keeps the focus, an icon glyph, a generated ID, and aria-hidden on the full page. The test makes sure that the recording captures the recipient, subject and body, that the check text contains parameters, and that the run passes with a new {now} value.

The tests use their own ports (8765 and 8767) and a temporary data folder. Thus they do not change the data of the operator.